materials_05_03_2026

Вот как теперь устроены ошибки “от и до” в твоём сервисе.

Слои и кто что кидает

1) Repos (FS/S3/MEM, Mongo)

Эти файлы **не знают про HTTP**. Они кидают **типизированные ошибки репозиториев**:

Хранилища (файлы/MinIO/память)

- `StorageLocationError` — некорректная location (например, для fs нет `path`, для s3 нет `bucket/key`) → это баг/неконсистентность → **500**
- `StorageNotFoundError` — объекта реально нет (файл/тайл/ключ в s3) → **404**
- `StorageIOError` — проблемы IO (права, сеть, MinIO, диск) → **500**

Метаданные (Mongo)

- `MetadataDBError` — Mongo упал/timeout/ошибка запроса → **500**
- `MetadataConflictError` — конфликт уникальности/гонка → **409**
- `MetadataDataError` — в базе лежит битый документ → **500**

2) Services (use-cases)

Сервисы дают **бизнес-смыслы**, а не “технические эксепшны”:

- `UseCaseValidationError` — плохие входные параметры (tile_size, fmt, файл не картинка) → **400**
- `UseCaseNotFoundError` — объект/метаданные/манифест не найдены → **404**
- `UseCaseConflictError` — конфликт (например UUID уже существует при `on_conflict=error`) → **409**

При этом storage/mongo ошибки из репо сервисы обычно **не прячут** — они просто поднимаются вверх (иногда сервис делает rollback и потом пробрасывает дальше).

3) API Routes (FastAPI роутеры)

Роуты теперь **почти ничего не ловят**.

Они:

- вызывают сервисы/репозитории
- возвращают успешный ответ
- не формируют `HTTPException` вручную (почти везде)

Исключение: “мягкие сценарии”, где ты сознательно хочешь вернуть **успех с предупреждением** (как в `ingest2` для enqueue jobs) — там можно ловить и превращать в `job_info`.

Как ошибка превращается в HTTP

Глобальные handlers

В `exception_handlers.py` ты регистрируешь обработчики:

- один handler на `StorageError`
- один handler на `MetadataError`
- один handler на `UseCaseError`

FastAPI делает так:

1. ошибка вылетает из repo/service вверх
2. роут её не ловит
3. FastAPI ловит её на уровне приложения
4. находит подходящий `@app.exception_handler(...)`
5. handler выбирает статус (404/400/409/500) и возвращает `JSONResponse`

Примеры “как теперь работает”

Пример A: `GET /metadata/images/{uuid}` (скачивание оригинала)

- если метаданных нет → `UseCaseNotFoundError("Метаданные...")` → **404**
- если мета есть, но файла/ключа нет в storage → `StorageNotFoundError(...)` → **404**
- если MinIO/диск упал → `StorageIOError(...)` → **500**

Пример B: `POST /ingest/...`

- если `uuid` уже есть и `on_conflict=error` → `UseCaseConflictError("UUID уже существует...")` → **409**
- если файл не изображение (probe) → `UseCaseValidationError("Файл не является корректным изображением...")` → **400**
- если Mongo недоступен при upsert → `MetadataDBError(...)` → **500** (и сервис делает rollback в storage)

Пример C: `GET /tiles/{uuid}/{z}/{y}/{x}`

- если манифеста нет → `UseCaseNotFoundError("Манифест тайлов не найден")` → **404**
- если координаты вне диапазона → `UseCaseNotFoundError("Тайл вне диапазона")` → **404**
- если тайл реально отсутствует в storage → `StorageNotFoundError("Тайл не найден")` → **404**

Главное “зачем так”

- **Repos**: чистая инфраструктура, без HTTP.
- **Services**: бизнес-смысл ошибок (валидация/конфликт/не найдено).
- **Routes**: тонкие, без try/except.
- **HTTP**: всё централизовано в одном месте (`exception_handlers.py`).